Reverie

An app that learns what you like to watch and picks what is next

Where we are right now | 9 days until we present (July 2)

What is Reverie, in plain words

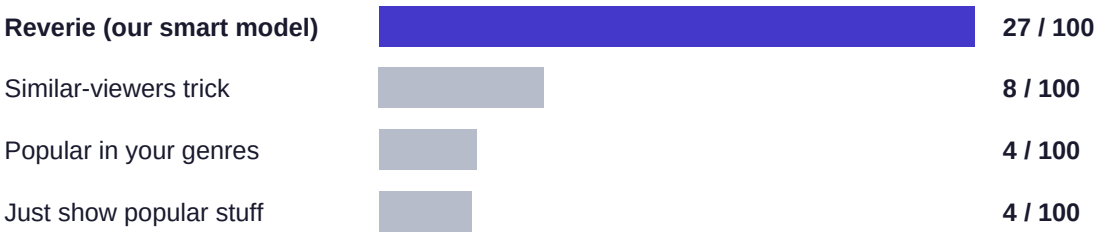
You give it a few movies and shows you have watched. It learns your taste from what you watched and in what order, then suggests what to watch next, both movies and TV. If you reject a suggestion, it adjusts on the spot. Think of a friend who knows your taste and keeps getting better every time you react.

Is it working? The quick status

The AI model (the brain)	The website
Works well	Half built
Load your own Netflix / Letterboxd	**The slideshow for class**
Not started	Not started

How good is the brain?

Out of every 100 guesses, how often was the show you actually watched next inside our top 10 picks? Longer bar is better.



Reverie is about 6 times better than just showing popular shows, and about 3 times better than the next-best simple method. Note: these are practice scores. The official final score still needs one last clean test run.

What is done and what is left

Get the movie data ready	Done
Build the AI model	Done
Check it beats the simple methods	Done
Fine-tune the model and run the final test	In progress
Finish the website	In progress

Let people load their own Netflix / Letterboxd	Not started
Make the class slideshow	Not started

Two quick decisions we need to make

1. **Should we add the popularity-fix trick, or remove it from our notes?**
   What it is: a small adjustment that stops the model from leaning on blockbusters everyone has already seen, so the picks feel more personal. Our written notes say we use it, but it is not actually in the code right now, so the notes and the code disagree.
   Why it matters: the professor reads our code out loud in class. Claiming something the code does not do is an easy thing to get caught on.
   Suggestion: Lea decides this week. Either add it for real (a small amount of work) or delete the line from our notes. Settle it before Stephan locks the final setup, because it changes the model.

2. **Do we also test the smaller version of the movie dataset?**
   What it is: we trained on the big dataset (about 1 million ratings). There is also a small version (about 100,000). Running both would let us say bigger data gave better results.
   The catch: the two datasets are stored in different file formats, so it means extra coding, and we already expect the big one to win. We chose the big one on purpose because the small one is too thin for this kind of model.
   Suggestion: Skip it. Add one sentence to the slideshow explaining why we chose the big dataset.

Who does what

Amaly | Data and bring-your-own-history

Can start now, nothing to wait on. Em and Cecile are waiting on your importers for the demo, so this is on the critical path.

1. **Get the movie data ready**
   - Make sure the big movie-ratings file loads with no errors
   - Write down which movies we kept and why, so the slideshow can explain it
2. **Build the part that reads someone's own history**
   - Read a Letterboxd file (movies a person rated) and match each one to ours
   - Read a Netflix file (shows a person watched, plus their thumbs)
3. **Add the TV show list**
   - Load a list of 3,000 TV shows with their genres
   - Match the Spanish Netflix titles to English for about 20 to 30 shows for the demo
   - Never upload anyone's personal files to GitHub

Stephan | The AI model

Can start now. Lea and Cecile are waiting on your locked setup, so lock it as early as you can.

1. **Try different settings and see what works best**
   - How far back the model looks (how many past shows it reads)
   - How big the model is
   - Two model types (GRU vs LSTM); pick the better one
2. **Lock in the best simple setup**
   - Choose the smallest setup that is still basically the best, and write it down so everyone uses the same one

3. **Be ready to explain the model**
   - The professor asks about the code live, so know what every part does

Lea | Scoring and proof

Can write the test code now. The official final test waits until Stephan locks the setup.

1. **Run the one official final test**
   - Use the locked setup on data we have never touched before
   - Repeat it 3 or more times so the result is not luck
   - Show it really beats the simple methods, with the margin of error
2. **Sort out the popularity-fix trick**
   - Either add it to the code or remove it from our notes (see decision 1 above)
3. **Make the not-cheating chart**
   - Show the model does well on new data, not just memorized answers

Em | The website

Can start the main wiring now against the current model. The load-your-own-history feature waits on Amaly's importers.

1. **Hook the website up to the real model**
   - Type in some shows, get real picks back, never fake or pre-typed ones
2. **Make the thumbs-down work**
   - Reject a pick and the list instantly changes
3. **Make the taste chart real**
   - The little taste graphic uses real output from the model

Cecile | Slideshow and the story

Can start the slides and business case now. Saving the final model waits on Stephan. The results slide waits on Lea. The live demo waits on Em and Amaly.

1. **Save the final model so the app can use it**
   - After Stephan locks the setup, train it on all the data and export it
2. **Build the 15-minute slideshow**
   - Cover the problem, our data, the model, results, the live demo, and why it matters for business
3. **Make a backup recording of the demo**
   - In case the live demo breaks, we still have a video to play
4. **Run the practice rounds**
   - Time the talk and rehearse the likely questions with the team

---

Code lives at github.com/em-ech/reverie. One laptop runs all the training so the numbers stay comparable.